Build the first physical station

A handoff guide for a teammate. Follow the pages in order; the two diagnostic sketches are complete and copyable.

Goal: get a real MPU-6050 detected, prove its readings change with movement, then upload the SmartRep firmware and count physical reps in the app. Start with USB; the ESP32 is optional.

What you are building

Sensor on the dumbbell → Arduino UNO R4 Minima → USB data cable → laptop bridge → phone app and instructor dashboard. The Arduino measures motion. The laptop estimates orientation, calibrates the exercise, counts reps and saves sets.

Parts and software

Item	What to prepare
Arduino	UNO R4 Minima. These instructions are not for an ESP32 or UNO R4 WiFi.
Motion sensor	MPU-6050 breakout, commonly GY-521. Confirm the actual module and its supply/logic ratings.
Signal translation	Bidirectional I2C level converter suitable for 5 V to 3.3 V, with pull-ups on both sides. Needed unless the breakout explicitly supports 5 V I2C.
Basic wiring	Breadboard, short jumpers, USB-C DATA cable, laptop. Multimeter recommended.
Optional later	Common-cathode RGB LED plus three 1 kohm resistors; ESP32 modem only after USB passes.
Software	Arduino IDE 2.x and Arduino UNO R4 Boards package; SmartRep project on the laptop.

Work in stages

1. Confirm power and wire the sensor (page 2). **2.** Upload Test A and find its I2C address (page 3). **3.** Upload Test B and verify motion readings (pages 4-5). **4.** Upload production firmware and calibrate the app (page 6). **5.** Mount the sensor and record a ten-rep trial (pages 7-8).

Verification: both test sketches compile for UNO R4 Minima with Arduino Renesas UNO core 1.6.0. No physical board or sensor has been tested by the author. Compilation does not establish correct wiring or real-world accuracy.

Wire power and I2C first

Unplug USB before changing wiring. Use short jumpers for this first test. References [1]-[3] are on page 8.

Power decision: the Arduino uses 5 V logic; the bare MPU-6050 is a lower-voltage device. Check your exact breakout documentation. A regulator on a module does not automatically make SDA/SCL 5 V tolerant. If the module is unidentified, stop and send clear photos of both sides before powering it.

The following is the conservative arrangement for a breakout confirmed to accept **3.3 V power and 3.3 V I2C**. Use an I2C-rated bidirectional converter, not the inverting UART circuit intended for the optional ESP32.

From	Connect to
Arduino 5V	Converter HV reference only in this 3.3 V sensor arrangement.
Arduino 3.3V	Converter LV reference AND sensor breakout VCC, if rated for this input.
Arduino GND	Converter GND AND sensor GND; all grounds common.
Arduino A4 / SDA	Converter HV1.
Converter LV1	Sensor SDA.
Arduino A5 / SCL	Converter HV2.
Converter LV2	Sensor SCL.
Sensor AD0	Sensor GND. This selects I2C address 0x68.
Sensor INT, XDA, XCL	Leave disconnected for this project.

If the breakout requires a documented 5 V VCC input: follow that module specification for VCC while keeping the sensor-side I2C logic correctly level shifted. Do not guess from a generic GY-521 label, and do not power a bare sensor chip from 5 V.

Before USB: check the actual pin labels, no short between supply and ground, correct HV/LV sides, and continuous ground. A4/SDA and A5/SCL are the Arduino signal pins; numbered converter channels can vary by module.

Test A find the sensor

Create a new Arduino sketch and replace its contents with this entire code block. File: smartrep_i2c_scan.ino

IDE setup: install Arduino IDE 2.x from arduino.cc/en/software. In Boards Manager install **Arduino UNO R4 Boards**. Select **Arduino UNO R4 Minima**, choose its USB port, then click Upload. Open Serial Monitor at **115200 baud**. No extra sensor library is needed.

```
// TEST A: UNO R4 Minima I2C scanner. Serial Monitor: 115200 baud.
// Standalone diagnostic only; do not run the SmartRep bridge with this sketch.
#include <Wire.h>
uint32_t nextScan = 0;
void setup() {
  Serial.begin(115200);
  pinMode(LED_BUILTIN, OUTPUT);
  Wire.begin();
  Wire.setClock(100000);
#if defined(ARDUINO_UNOR4_MINIMA) || defined(WIRE_HAS_TIMEOUT)
  Wire.setWireTimeout(25000, true);
#endif
}
void loop() {
  digitalWrite(LED_BUILTIN, (millis() / 500) % 2);
  if ((int32_t)(millis() - nextScan) < 0) return;
  nextScan = millis() + 5000;
  Serial.println("Scanning I2C...");
  uint8_t found = 0;
  for (uint8_t address = 8; address < 120; address++) {
    Wire.beginTransmission(address);
    uint8_t error = Wire.endTransmission();
    if (error == 0) {
      Serial.print("Found device at 0x");
      if (address < 16) Serial.print('0');
      Serial.println(address, HEX);
      found++;
    }
  }
  Serial.print("Devices found: ");
  Serial.println(found);
  Serial.println("MPU-6050 normally appears at 0x68 or 0x69.");
  Serial.println();
}
```

Pass: expect `Found device at 0x68` every five seconds. **0x69** means AD0 is high; ground AD0 to match the SmartRep firmware. No devices, many unexpected addresses, or intermittent results mean stop and check wiring before continuing.

Test B verify motion readings

Copy all code below into a new sketch. Upload to UNO R4 Minima. Serial Monitor: 115200 baud.

```
// TEST B: UNO R4 Minima + MPU-6050. Serial Monitor: 115200 baud.
// Change ADDRESS to 0x69 only if Test A found 0x69.
#include <Wire.h>
const uint8_t ADDRESS = 0x68;
bool ready = false;
uint32_t nextSample = 0, nextInit = 0;
bool writeReg(uint8_t reg, uint8_t value) {
  Wire.beginTransmission(ADDRESS);
  Wire.write(reg); Wire.write(value);
  return Wire.endTransmission() == 0;
}
bool readReg(uint8_t reg, uint8_t count, uint8_t *data) {
  Wire.beginTransmission(ADDRESS); Wire.write(reg);
  if (Wire.endTransmission(false) != 0) return false;
  if (Wire.requestFrom(ADDRESS, count) != count) return false;
  for (uint8_t i = 0; i < count; i++) data[i] = Wire.read();
  return true;
}
int16_t signed16(const uint8_t *p) {
  return (int16_t)(((uint16_t)p[0] << 8) | p[1]);
}
bool initSensor() {
  uint8_t who = 0;
  if (!readReg(0x75, 1, &who)) {
    Serial.println("FAIL: cannot read sensor. Run Test A.");
    return false;
  }
  Serial.print("WHO_AM_I = 0x"); Serial.println(who, HEX);
  if (who != 0x68) {
    Serial.println("FAIL: unexpected chip ID; confirm sensor model.");
    return false;
  }
  bool ok = writeReg(0x6B, 0) && writeReg(0x1A, 3) &&
            writeReg(0x1B, 8) && writeReg(0x1C, 8);
  if (ok) Serial.println("PASS: MPU-6050 configured; accel g, gyro dps.");
  else Serial.println("FAIL: configuration write failed.");
  return ok;
}
void setup() {
  Serial.begin(115200); pinMode(LED_BUILTIN, OUTPUT);
  Wire.begin(); Wire.setClock(100000);
#if defined(ARDUINO_UNOR4_MINIMA) || defined(WIRE_HAS_TIMEOUT)
  Wire.setWireTimeout(25000, true);
#endif
}
void loop() {
  digitalWrite(LED_BUILTIN, ready ? HIGH : (millis() / 250) % 2);
  if (!ready) {
    if ((int32_t)(millis() - nextInit) < 0) return;
    nextInit = millis() + 3000; ready = initSensor(); return;
  }
  if ((int32_t)(millis() - nextSample) < 0) return;
  nextSample = millis() + 100;  // Readable 10 Hz diagnostic output.
  uint8_t data[14];
  if (!readReg(0x3B, 14, data)) {
    Serial.println("FAIL: sensor read; check cable and power.");
    ready = false; return;
  }
  Serial.print("accel_g: ");
  for (uint8_t i = 0; i < 3; i++) {
    Serial.print(signed16(data + 2 * i) / 8192.0, 3);
    Serial.print(i == 2 ? " | gyro_dps: " : ", ");
  }
  for (uint8_t i = 0; i < 3; i++) {
    Serial.print(signed16(data + 8 + 2 * i) / 65.5, 2);
    if (i < 2) Serial.print(", ");
  }
  Serial.println();
}
```

Interpret Test B and fix failures

The diagnostic sketches run by themselves. They do not count reps and do not implement the app protocol.

Expected startup: `WHO_AM_I = 0x68`, then `PASS: MPU-6050 configured; accel g, gyro dps.` If Test A found address 0x69, change ADDRESS in Test B to 0x69; the chip ID still normally reads 0x68.

Three movement checks

Check	What should happen
Rest on a flat surface	One accelerometer axis should be approximately +1 g or -1 g; the others near 0 g. Which axis depends on mounting. Gyro values should be near 0 dps with small bias/noise.
Tilt slowly, then hold	The gravity reading transfers between accelerometer axes. Gyro values change while turning, then return near zero when held still.
Rotate in your hand	At least one gyro axis changes clearly. Values should not remain frozen, all zero, or persistently at the range limits.

Example only: `accel_g: 0.010, -0.020, 0.998 | gyro_dps: 0.20, -0.40, 0.10`. Your numbers will differ. Test B uses +/-4 g and +/-500 dps settings, prints about 10 lines/second, and does not remove gyro bias. SmartRep production firmware streams at 50 Hz.

Troubleshooting

Symptom	Next action
No USB port / upload fails	Try a known data cable, another USB port and the correct board selection. Close other serial programs. Confirm the board is UNO R4 Minima.
Gibberish text	Set Serial Monitor to 115200 baud.
Test A finds nothing	Check common ground, module supply rating, SDA/SCL order, converter HV/LV references and pull-ups. Check breadboard rail breaks.
Unexpected WHO_AM_I	Confirm the printed chip/module model. A different IMU or clone may need different firmware; do not treat any responding address as a pass.
Intermittent read failures	Shorten the cable, secure connections and check supply voltage. Keep the 100 kHz I2C setting. Do not extend the cable until short-wire tests pass.
Port busy / access denied	Close Serial Monitor, the SmartRep bridge and any other program using that port before uploading or running a standalone recorder.

Move from test code to SmartRep

After Test A and Test B pass, replace the diagnostic sketch with the production sketch supplied in the companion ZIP.

1 upload the production sketch

Open `firmware/smartrep_arduino/smartrep_arduino.ino` in Arduino IDE. Keep the folder and .ino filename together. Check the settings below, select UNO R4 Minima and upload. Only Wire is required; no third-party MPU library is used.

```
#define DEVICE_ID "clip-01"
#define SENSOR_MPU6050
```

In Serial Monitor, use **115200 baud** and **Newline** line ending. Send `HELLO` for identity, `START` for raw JSON samples, `PING` for a pong and `STOP` to stop. START expires after 10 seconds without a command. HELLO alone does not prove the sensor works; START must produce changing samples.

2 close Serial Monitor and start the bridge

The teammate needs the full project and its installed Python environment. The private GitHub repo is **aaronkoolkoder/ece198-projects**; the owner must grant access or supply the project files. The guide ZIP contains firmware, not the entire app. Stop the simulated bridge before using port 8000.

macOS / Linux, from the project folder:

```
cd bridge
.venv/bin/python -m smartrep --db smartrep-hardware.db
```

Windows PowerShell, from the project folder:

```
cd bridge
.\.venv\Scripts\python.exe -m smartrep --db smartrep-hardware.db
```

If auto-detection misses the board, append `--port COM4` on Windows, or `--port /dev/cu.usbmodemXXXX` on macOS, using the real port. Do **not** add --simulate. The separate hardware DB preserves the simulation and avoids reusing fake motion references.

3 calibrate and run a set

Open **http://127.0.0.1:8000/** on the laptop. Continue to the app and choose Mary, ID **123129**. Check Instructor view: clip-01 should be online at Dumbbells 5 lb. Open **Staff calibration**, PIN **0000**, and choose Bicep curl. Hold still for at least half a second, do three controlled full reps, return to the start and save. Then choose the 5 lb station, Bicep curl and Start set.

If a teammate has not installed Python dependencies, follow the project README first. The phone can open the laptop LAN address on a shared hotspot/router. School WiFi can block that route; a temporary tunnel is another option. The laptop must stay awake.

Mounting status light and later WiFi

First prove the short-wire USB station. Add optional parts one at a time, repeating the sensor test after each change.

Mount the sensor so it cannot slip

Attach only the sensor securely to the dumbbell; keep the Arduino and breadboard in a stationary station box. Add strain relief at the sensor and box. Keep wires away from fingers and moving equipment. Test unloaded or with a light object first. A shifting sensor invalidates the saved motion reference.

After the short-wire setup works, trial the planned 1-1.5 m cable at 100 kHz. Longer I2C wiring is sensitive to capacitance, pull-ups and interference. Start short and increase gradually; a planned length is not a guarantee of reliable operation. Re-run the motion test and watch for read errors before using it for a demo.

Optional RGB station light

Use a **common-cathode** RGB LED. Identify its pins from its datasheet; do not assume a leg order. Give each color its own **1 kohm series resistor**. The production firmware works without this LED attached.

Arduino pin	LED connection
D9	Through 1 kohm to RED anode.
D10	Through 1 kohm to GREEN anode.
D11	Through 1 kohm to BLUE anode.
GND	LED common cathode.

Production indications: red blink = sensor error; fast blue = WiFi connecting; blue pulse = bridge idle; dim green = streaming; brief red every three seconds = no bridge. The built-in LED provides a heartbeat. The phone supplies rep sounds and vibration.

Optional ESP32 after USB passes

An ESP32 is needed only to remove the USB data cable from the Arduino to the laptop. The UNO R4 Minima itself has no WiFi. The prepared modem lives in `firmware/smartrep_esp32_wifi/`. It needs an ESP32 board package, WebSockets by Markus Sattler, and a local config.h made from config.example.h.

> The prepared modem assumes **two inverting UART voltage translators**: Arduino D1 TX to ESP32 GPIO16 RX, and ESP32 GPIO17 TX to Arduino D0 RX, with common ground. Its UART invert flag is true. This is a separate electrical circuit from the bidirectional I2C converter. Do not connect 5 V TX directly to an ESP32 input. Have the actual translator schematic checked before powering it.

Attach the WROOM-32U external antenna. Keep ESP32 USB power and the Arduino-to-ESP32 5 V feed from powering the rail simultaneously. The ESP32 sketch has not been compiler-verified in this environment because its toolchain download failed. Treat WiFi as a separate bring-up step; it is not required for the USB milestone.

Handoff checklist and references

Share this PDF and SmartRep_Hardware_Handoff.zip. Both diagnostic code blocks match the .ino files in the ZIP.

Record these before handing it back

Check	Record result
Board and sensor	UNO R4 Minima confirmed; exact sensor breakout model and photo of both sides.
Power and translation	Documented sensor VCC input; converter HV/LV and ground checked.
Test A	Found address: _____ Stable over 30 seconds: yes / no
Test B	WHO_AM_I: _____ At rest near 1 g total: yes / no
Motion and cable	Acceleration and rotation change; no recurring read errors: yes / no
Production firmware	clip-01 identity received; START streams samples: yes / no
App	Hardware mode, station online, personal calibration saved: yes / no
Three 10-rep trials	Human / app: 10 / ____ 10 / ____ 10 / ____

Target: at least 9 of 10 reps correctly counted in each of three trials, while also checking that resting motion does not add reps. Try a few deliberate short attempts and confirm they are flagged rather than added to completed reps. This target has not yet been established on real hardware.

What to send to the project owner

Send the checklist, sensor and wiring photos, board port, module supply information, test output, and any error messages. If the sensor moves on the dumbbell, recalibrate. If the short-wire setup works but the long cable fails, report that distinction instead of changing multiple parts at once.

Reference documents

[1] **Arduino UNO R4 Minima datasheet** - 5 V operation, USB power, A4/A5 I2C and board pinout. Official Arduino datasheet.
[2] **InvenSense MPU-6000/6050 product specification** - chip supply and logic ratings, I2C addressing and sensor scales. This is the manufacturer document hosted by a distributor. MPU-6050 datasheet.
[3] **Arduino Wire reference** - board I2C pin mapping and API. Arduino Wire reference.
[4] **Arduino IDE download**. arduino.cc/en/software.

Compiled on November 5, 2026 for arduino:renesas_uno:minima, core 1.6.0. Test A: 45,140 bytes flash / 5,096 bytes global RAM. Test B: 46,496 bytes flash / 5,104 bytes global RAM. The production sketch had previously compiled for R4. Physical wiring, sensor accuracy and long-cable behavior remain to be verified.